Industr.IA

E-book

Edge AI: A decisão de Deployment como compromisso de produto



Curso base de IA | Unidade M4

Prof.: Arthur Nascimento

SUMÁRIO

1. A decisão de posicionamento no continuum 3
2. A decisão de compressão e runtime 7
3. A estrutura de custo total 12
4. Conclusão 16

Unidade M4

1. A decisão de posicionamento no continuum

Esta seção tem por objetivo que você aplique a primeira das três decisões da lente do deployment, o posicionamento da inferência no continuum cloud-edge, usando uma heurística explícita e produzindo uma justificativa registrável; ela pertence à camada decisória do módulo. Os tópicos anteriores descreveram o continuum e as motivações que deslocam a inferência em direção à ponta. Aqui a pergunta muda de natureza: não se trata mais de saber quais posições existem, e sim de decidir qual posição este projeto deve assumir, e de deixar por escrito o porquê. A diferença parece sutil, mas é ela que separa um time que conhece o vocabulário de um time que sustenta uma escolha diante de stakeholders.

A heurística que organiza a decisão cabe em uma frase: vale Edge quando a latência é compromisso de produto, quando a privacidade é compromisso regulatório, ou quando a conectividade não é assumível no ponto de operação. A força da frase está em duas palavras, compromisso e assumível. Cada um dos três gatilhos é um teste de natureza, e não de intensidade: ele não pergunta se latência menor seria desejável, porque desejável ela sempre é; pergunta se a latência está amarrada a algo que não se negocia. É essa exigência de amarração que impede a heurística de degenerar em preferência, e é ela que os próximos parágrafos destrincham gatilho a gatilho.

O primeiro gatilho testa a latência como compromisso de produto. O teste tem duas partes. A primeira é nomear a amarração: um tempo de ciclo de linha que a esteira impõe, um SLA assinado com o cliente, um requisito de segurança de operação como o alerta de fadiga ou a parada de emergência. Se a resposta atrasada invalida o produto, viola contrato ou expõe pessoas, a latência é compromisso; se apenas melhora a experiência, ainda não é. A segunda parte é quantificar: qual é o orçamento de latência de ponta a ponta, e quanto dele a viagem de rede até a nuvem consumiria no pior caso plausível, não na média. Quando a viagem sozinha estoura o orçamento, nenhuma otimização de servidor recupera o tempo perdido no trajeto, e o gatilho dispara. Quando sobra folga com margem, o gatilho não dispara, ainda que a resposta local fosse mais rápida.

O segundo gatilho testa a privacidade como compromisso regulatório, e inclui a vizinha soberania de dados. O teste é direto: existe base legal, cláusula contratual ou norma setorial que impeça o dado bruto de sair do local, da organização ou da jurisdição? No contexto brasileiro, a LGPD é a âncora: os princípios de necessidade e de minimização favorecem processar o dado pessoal localmente e transmitir apenas o derivado agregado, e certos contratos com órgãos públicos e setores regulados transformam esse favorecimento em exigência. O gatilho dispara quando a obrigação é nomeável, com artigo,

cláusula ou norma identificados; não dispara quando a privacidade local é apenas uma preferência genérica de arquitetura. O registro da decisão, nesse caso, precisa dizer três coisas: qual dado bruto fica, qual obrigação o retém, e qual derivado pode subir.

O terceiro gatilho testa se a conectividade é assumível. Assumível significa que o ponto de operação garante rede estável, com banda suficiente, durante todo o período em que a solução precisa funcionar. Campo aberto, embarcações, subsolo de mina, equipamentos móveis e plantas com política de rede isolada respondem não com facilidade. A nuance está no caso intermediário: a rede existe, mas cai algumas vezes por semana, ou satura nos horários de pico. Se a função principal da solução não pode parar junto com a rede, a conectividade não é assumível para essa função, e a inferência precisa ser local ainda que a sincronização com a nuvem ocorra quando o enlace volta. Degradar com elegância, aqui, é sinônimo de inferir localmente.

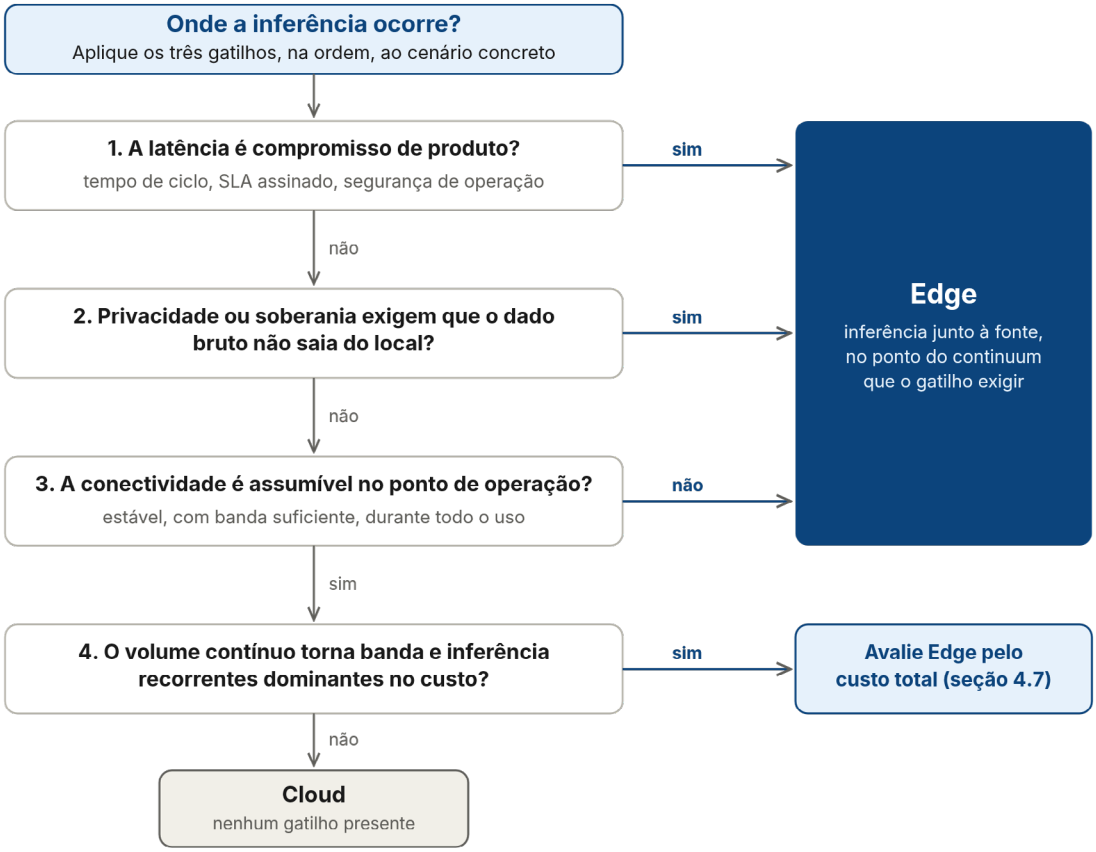
A ordem em que os gatilhos aparecem na heurística não é a ordem de importância, e sim a ordem de custo de verificação. O teste de latência é o mais barato: o orçamento de ponta a ponta e o tempo de viagem da rede são mensuráveis cedo, com um protótipo mínimo ou até com aritmética de rede. O teste regulatório exige leitura jurídica ou contratual, que tem fila e prazo próprios, e por isso deve ser disparado no início do projeto ainda que a resposta chegue depois. O teste de conectividade exige levantamento de campo no ponto real de operação, medindo estabilidade e banda ao longo de dias típicos, e não numa visita pontual. Na prática os três correm em paralelo; o que a heurística ordena é o registro: cada teste entra na justificativa com a sua evidência, mesmo quando a resposta é não.

Vale explicitar também o que a decisão de posicionamento produz, porque o resultado não é o binário edge ou cloud. Quando um gatilho dispara, ele determina o deslocamento mínimo ao longo do continuum visto no tópico 1 do ebook anterior, e pontos diferentes satisfazem gatilhos diferentes. Um orçamento de latência de poucos milissegundos exige inferência no edge ou no endpoint, junto à fonte; uma obrigação de que o dado não saia da planta pode ser satisfeita na camada fog, num servidor local que atende várias linhas, sem exigir um dispositivo por câmera; uma conectividade intermitente aceita a fog local com sincronização oportunista. Escolher o ponto mais distante da nuvem do que o gatilho exige encarece a solução sem ganho; escolher um ponto aquém do gatilho descumpre o compromisso. O registro correto nomeia o ponto, e não apenas o lado.

E quando nenhum dos três gatilhos dispara? A cloud não vence por padrão. Entra uma quarta pergunta, de natureza econômica: o volume contínuo de inferências é alto o bastante para que banda e inferência recorrentes dominem o custo ao longo do tempo? Se sim, o edge volta ao jogo pela via do custo total, que é o terceiro eixo da decisão e ocupa o último tópico deste ebook. Se não, a cloud é a resposta adequada, e registrá-la como

decisão tomada, e não como omissão, tem valor: protege o projeto de refazer a discussão a cada reunião. A figura a seguir organiza a sequência completa.

Figura 1 - Árvore de decisão Edge versus Cloud, os três gatilhos e a pergunta econômica.



Toda saída Edge segue para as decisões de compressão e runtime (seção 4.6) e de custo total (seção 4.7).

Fonte: autor.

Figura intitulada "Figura 1. Árvore de decisão Edge versus Cloud, os três gatilhos e a pergunta econômica". O diagrama apresenta uma árvore de decisão composta por caixas conectadas por setas, orientando a escolha entre executar a inferência em Edge ou em Cloud. No topo, uma caixa azul-clara traz a pergunta "Onde a inferência ocorre?", acompanhada da orientação: "Aplique os três gatilhos, na ordem, ao cenário concreto."
A primeira decisão pergunta: "1. A latência é compromisso de produto?", exemplificando tempo de ciclo, SLA assinado e segurança de operação. Se a resposta for "sim", o fluxo segue diretamente para uma caixa azul-escura intitulada "Edge", descrita como "inferência junto à fonte, no ponto do continuum que o gatilho exigir". Se a resposta for "não", o fluxo continua para a segunda decisão.
A segunda pergunta é: "2. Privacidade ou soberania exigem que o dado bruto não saia do local?". Se a resposta for "sim", a decisão também direciona para Edge. Caso contrário, o fluxo segue para a terceira decisão.
A terceira pergunta é: "3. A conectividade é assumível no ponto de operação?", considerando uma conexão estável, com banda suficiente e disponível durante todo o uso. Se a resposta for "não", a recomendação permanece Edge. Se a resposta for "sim", o fluxo segue para a quarta e última decisão.
A quarta pergunta é: "4. O volume contínuo torna banda e inferência recorrentes dominantes no custo?". Se a resposta for "sim", o diagrama recomenda "Avalie Edge pelo custo total (seção 4.7)". Se a resposta for "não", o fluxo termina em uma caixa cinza intitulada "Cloud", acompanhada da observação "nenhum gatilho presente".
Na parte inferior da figura, uma observação informa que toda saída para Edge segue para as decisões de compressão e runtime (seção 4.6) e de custo total (seção 4.7). O fluxograma utiliza caixas em tons de azul e cinza conectadas por setas, representando a sequência lógica das decisões para definir o local mais adequado para a execução da inferência. Na parte inferior consta a indicação "Fonte: autor."

Convém dizer como usar a árvore sem transformá-la em automatismo. Ela ordena as perguntas, mas não substitui a análise de cada uma: responder sim ou não exige a evidência que os parágrafos anteriores pediram, o número de latência medido, a cláusula localizada, o mapa de conectividade do ponto de operação. Gatilhos múltiplos reforçam a decisão, como no caso da transportadora visto no tópico 1 do ebook anterior, em que latência, privacidade e conectividade apontavam juntos para o edge. E gatilhos parciais, que valem para uma parte do fluxo e não para o todo, costumam levar à divisão da solução entre pontos do continuum, em vez de forçar uma resposta única.

GLOSSÁRIO

Arquitetura híbrida

Divisão deliberada da inferência entre pontos do continuum: a parte da solução amarrada a latência, privacidade ou operação offline roda no edge, e a parte que tolera espera ou exige um modelo maior roda na cloud. É o resultado frequente da árvore de decisão quando os gatilhos valem para parte do fluxo, não para o todo, e exige registrar qual parte roda onde e por quê.

O produto desta seção é uma decisão registrada, não uma intuição compartilhada. O registro responde os três testes com a evidência de cada resposta e, quando nenhum dispara, documenta a passagem para a análise de custo. É esse registro que o instrumento de decisão do módulo, o template T4.1, estrutura em forma de matriz, e é ele que o encontro presencial de desenho da solução espera encontrar pronto, e não em elaboração.

Um último cuidado do primeiro eixo é lembrar que gatilhos têm prazo de validade, porque compromissos mudam. Um SLA renegociado pode afrouxar o orçamento de latência; uma anonimização aplicada na fonte pode retirar o dado do alcance da obrigação que o retinha; a chegada de um enlace dedicado pode tornar assumível a conectividade que não era. Nada disso desfaz a decisão tomada, mas muda a resposta que seria tomada hoje, e é por isso que o registro guarda o teste, e não só o resultado: quando o compromisso que sustentava um gatilho cai, a matriz aponta exatamente o que reavaliar. O mesmo registro serve à defesa da decisão diante de stakeholders não técnicos, porque traduz a escolha em linguagem de risco e de obrigação, o SLA que seria violado, a cláusula que seria descumprida, o turno que ficaria descoberto, em vez de vocabulário de arquitetura.

Exemplo aplicado

Uma mineradora quer detectar desgaste de mancal em britadores de uma mina a céu aberto, por análise acústica contínua. O teste de latência não dispara: um alerta em minutos basta para programar a manutenção. O de privacidade não dispara: dado de vibração de máquina não é pessoal nem sigiloso por contrato. O de conectividade dispara: o enlace de satélite da mina é intermitente e a detecção precisa cobrir o turno inteiro. A inferência roda no edge, num gateway junto aos britadores, e um resumo diário sobe quando há enlace. Um único gatilho, bem evidenciado, decidiu o posicionamento.

> ⚠ ATENÇÃO
>
> A confusão decisória típica desta etapa é tratar gatilho como preferência somável: escolher edge porque latência menor é sempre melhor, ou porque manter dados localmente parece mais seguro, sem apontar um compromisso nomeável.
>
> A consequência é assumir o custo permanente de operar dispositivos em campo, com frota, atualização e manutenção, para honrar um compromisso que ninguém assinou. O erro inverso também ocorre: descartar o edge porque a nuvem é o padrão da empresa, ignorando um SLA que a viagem de rede não consegue cumprir. Nos dois casos, falta o teste, não a opinião.

Fixado o ponto do continuum, a decisão seguinte é fazer o modelo caber nele. A próxima seção conduz a escolha de técnica de compressão e de runtime como uma decisão condicionada ao alvo, e não como um cardápio de técnicas.

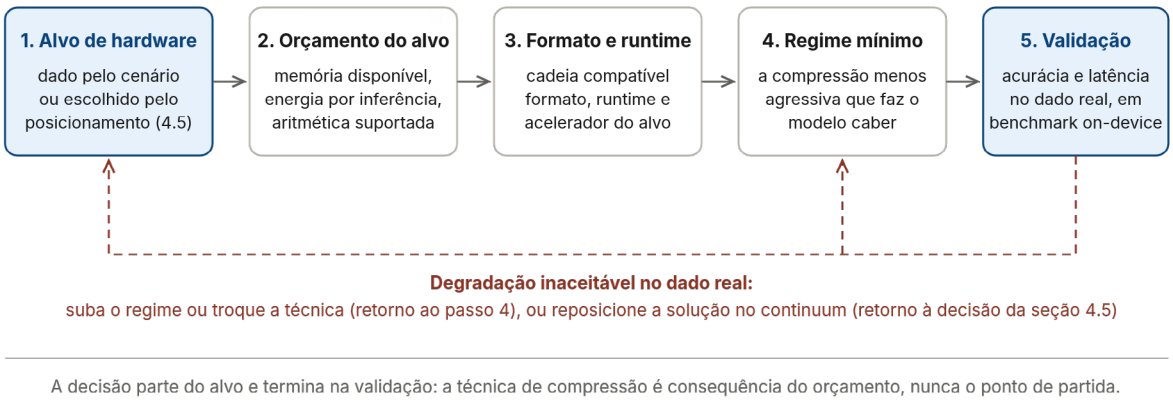
2. A decisão de compressão e runtime condicionada ao alvo

Esta seção tem por objetivo que você conduza a segunda decisão da lente do deployment, a escolha da técnica de compressão e do runtime apropriados ao alvo, tratando o compromisso entre acurácia, tamanho e latência como decisão registrada; ela pertence à camada decisória do módulo. O tópico 3 do ebook anterior apresentou as três famílias de compressão e o que cada uma negocia; já o tópico 2 separou formato, runtime e hardware. Aqui muda o verbo: de reconhecer para escolher. E a primeira coisa a fixar sobre a escolha é a ordem em que ela acontece.

O erro clássico é começar pela técnica, anunciando que o modelo será quantizado como se a técnica fosse a decisão. A decisão começa no alvo e termina na validação, em cinco passos. Primeiro, fixar o alvo de hardware, que ou é dado pelo cenário, o gateway que já existe na planta, o celular do usuário, o acelerador homologado pela empresa, ou é escolhido como consequência direta do posicionamento do tópico 1 deste ebook. Segundo, levantar o orçamento do alvo: a memória disponível para o modelo somado às ativações que ele gera, a energia por inferência que o ciclo de trabalho comporta, e a aritmética que o acelerador de fato suporta, lembrando da TPU de borda que só brilha com INT8, vista no tópico 2 do ebook anterior deste módulo. Terceiro, selecionar a cadeia de formato e runtime compatível com esse alvo, verificada cedo com o modelo real, porque é na conversão que os projetos tropeçam. Quarto, aplicar o regime de compressão mínimo que faz o modelo caber: começar pelo menos agressivo, FP16, depois

INT8, e só descer degraus quando a restrição obrigar. Quinto, validar acurácia e latência no dado real da aplicação, no dispositivo real. A figura a seguir organiza a sequência e o seu ponto de retorno.

Figura 2 - Sequência da decisão de compressão e runtime, do alvo à validação, com o retorno em caso de degradação inaceitável.



Fonte: autor.

Figura intitulada "Figura 2. Sequência da decisão de compressão e runtime, do alvo à validação, com o retorno em caso de degradação inaceitável". O diagrama apresenta um fluxo horizontal composto por cinco etapas conectadas por setas, descrevendo o processo de seleção e validação de técnicas de compressão para modelos de inteligência artificial. A primeira etapa é "1. Alvo de hardware", indicando que a decisão parte do hardware escolhido pelo posicionamento. Em seguida, a etapa "2. Orçamento do alvo" considera fatores como memória disponível, energia por inferência e aritmética suportada. A terceira etapa, "3. Formato e runtime", define a cadeia compatível entre formato do modelo, ambiente de execução e acelerador do dispositivo. A quarta etapa é "4. Regime mínimo", que orienta a selecionar a compressão menos agressiva capaz de fazer o modelo caber no dispositivo. Por fim, a quinta etapa, "5. Validação", verifica a acurácia e a latência utilizando dados reais e benchmark executado diretamente no dispositivo.
Abaixo da sequência principal, uma seta tracejada vermelha representa um caminho de retorno identificado como "Degradação inaceitável no dado real". Nessa situação, a orientação é subir o regime de compressão ou trocar a técnica, retornando ao passo 4, ou reposicionar a solução no continuum, retornando à decisão da seção 4.5. Na parte inferior, uma observação reforça que a decisão começa pela escolha do hardware e termina na validação, sendo a técnica de compressão uma consequência do orçamento disponível, e não o ponto de partida. O diagrama utiliza caixas retangulares em tons de azul e cinza, conectadas por setas horizontais, com um fluxo de retorno em vermelho indicando o processo iterativo de ajuste. Na parte inferior consta a indicação "Fonte: autor."

O segundo passo, o orçamento do alvo, admite uma estimativa de gabinete que antecipa boa parte do veredito antes de qualquer medição. A memória exigida é a soma de três parcelas: os pesos do modelo, que valem o número de parâmetros multiplicado pelos bytes por peso do regime, quatro em FP32, dois em FP16, um em INT8; o pico de ativações, que depende da arquitetura e do tamanho da entrada; e a sobrecarga do próprio runtime. Um modelo de 50 milhões de parâmetros em INT8 pesa cerca de 50 MB de pesos, e a pergunta passa a ser se as ativações e o runtime cabem no que sobra do alvo. Essa aritmética elimina combinações inviáveis em minutos e orienta em que regime começar, mas não substitui o benchmark do quinto passo: ela estima o que cabe, não o que a aplicação tolera em acurácia nem o que o pipeline entrega em latência.

Vale justificar o quarto passo, porque ele contraria um instinto comum. Se INT4 faz o modelo caber com o dobro de folga, por que parar em INT8? Porque cada degrau de compressão gasta uma parte do orçamento de erro da aplicação, e orçamento de erro gasto não volta. Guardar margem para o que ainda vem, o drift dos dados, os slices novos que a produção revela, a próxima versão do modelo, é prudência de projeto, não conservadorismo. O regime mínimo suficiente também torna a decisão auditável: fica claro qual restrição forçou cada degrau, e o que precisaria mudar para subir de volta. A

folga extra de INT4 só compra algo quando existe uma restrição concreta que a exige, e nesse caso ela deixa de ser folga e vira requisito.

O quinto passo merece o mesmo cuidado, porque é nele que a decisão se prova ou se desfaz. Três números mínimos, medidos no dispositivo alvo: a latência por inferência no pior caso relevante, e não na média, porque o compromisso de produto do tópico 1 deste ebook foi assumido para o pior caso; o pico de memória, modelo mais ativações, porque é o pico que derruba o processo; e a métrica de qualidade da tarefa, calculada no dado real da aplicação e comparada com o modelo sem compressão. A comparação por slice importa tanto quanto a agregada: a degradação da quantização não é uniforme, e um modelo que perde meio ponto na média pode perder cinco pontos num slice minoritário, exatamente o padrão que a lente dos dados do módulo M3 ensinou a procurar.

GLOSSÁRIO

Benchmark on-device

Medição de latência, memória e acurácia executada no dispositivo alvo, com o modelo no formato final e o dado da aplicação, em vez de no ambiente de desenvolvimento. É a evidência que valida uma decisão de compressão: números medidos em servidor, ou sobre dados de benchmark público, não transferem para a ponta, porque mudam o hardware, a aritmética e a distribuição dos dados ao mesmo tempo.

Com a sequência fixada, a escolha da técnica pode ser lida pela restrição que domina o caso. Quando a memória do alvo não comporta o modelo, a quantização é o primeiro caminho, do regime menos agressivo ao necessário. Quando o modelo cabe mas a latência estoura, a quantização se soma a um runtime que aproveite o acelerador do alvo, e o pruning estruturado entra se houver redundância clara na rede. Quando o alvo é ordens de grandeza menor que o modelo, comprimir não basta: o caminho é o knowledge distillation, treinando um aluno já dimensionado ao alvo, e em seguida quantizando o aluno, no encadeamento visto no tópico 3 do ebook anterior deste módulo. Quando a energia por inferência é o que limita, a combinação de acelerador de aritmética inteira com modelo quantizado tende a entregar mais inferências por watt. O quadro a seguir organiza essa leitura com o que verificar antes de fechar cada caminho.

Quadro 1 - A decisão de compressão lida pela restrição dominante do alvo.

Restrição dominante	Caminho indicado	O que verificar antes de fechar
A memória do alvo não comporta o modelo	quantização, do regime menos agressivo ao necessário	acurácia por slice no dado real; suporte do acelerador à aritmética escolhida
O modelo cabe, mas a latência estoura o orçamento	quantização somada a runtime com aceleração no alvo; pruning estruturado se houver redundância	latência de pior caso medida no dispositivo, não a média em servidor
O alvo é ordens de grandeza menor que o modelo	knowledge distillation para um aluno dimensionado ao alvo, depois quantização do aluno	custo de treinar o aluno cabe no cronograma; qualidade do aluno no dado real
A energia por inferência limita a operação	acelerador de aritmética inteira (NPU, TPU de borda) com modelo quantizado	inferências por watt medidas; autonomia sustentada no ciclo de trabalho real

Quadro 1 intitulado "A decisão de compressão lida pela restrição dominante do alvo", organizado em formato de tabela com três colunas: Restrição dominante, Caminho indicado e O que verificar antes de fechar. O quadro apresenta quatro cenários que orientam a escolha da estratégia de compressão de modelos de inteligência artificial conforme a principal limitação do ambiente de implantação.
Na primeira linha, quando a memória do dispositivo não comporta o modelo, recomenda-se realizar quantização, reduzindo a precisão dos cálculos até o nível menos agressivo necessário. Antes da adoção da solução, deve-se verificar a acurácia por fatia de dados, assegurando que a redução de precisão preserve a qualidade do modelo, além de confirmar o suporte do acelerador ou da aritmética utilizada.
Na segunda linha, quando o modelo cabe na memória, mas a latência excede o orçamento disponível, o caminho indicado é combinar quantização com poda de parâmetros e aceleração do ambiente, removendo estruturas redundantes quando existirem. A validação deve considerar a latência medida diretamente no dispositivo, avaliando o pior caso de execução, e não apenas a média obtida em ambiente de servidor.
Na terceira linha, quando o alvo é composto por grandes modelos menores do que o modelo original, recomenda-se aplicar knowledge distillation, treinando um modelo menor, chamado de aluno, a partir de um modelo maior, seguido da quantização desse modelo reduzido. Antes da implementação, é necessário verificar o custo de treinamento, confirmar se o modelo reduzido atende ao cronograma previsto e avaliar sua qualidade em dados reais.
Na quarta linha, quando a restrição está na inferência, limitando a operação, a recomendação é acelerar a execução utilizando aritmética inteira, como inteiros de oito bits (INT8), ou poda estrutural, combinando essas estratégias quando necessário. A etapa final consiste em verificar se a inferência atende aos requisitos de tempo, além de confirmar que os ganhos observados permanecem sustentados durante a operação em ambiente real.

Dentro da quantização, a escolha entre os dois modos nomeados no tópico 3 do ebook anterior também é decisão de cronograma, e a ordem racional é fixa. A quantização pós-treino vem primeiro, porque converte o modelo já treinado em horas e sem novo ciclo de treino: se ela passa na validação do quinto passo, a decisão fecha barata. O treino consciente de quantização entra apenas quando a pós-treino falha na validação e o regime não pode subir, porque ele recupera acurácia ao custo de um novo ciclo de treino, com dados, tempo de computação e calendário. Registrar essa ordem evita dois desperdícios simétricos: pagar um retreino que a conversão direta dispensaria, e insistir em regimes cada vez mais agressivos de pós-treino quando o retreino consciente resolveria com um único degrau.

Sobre a latência, um detalhamento evita medições enganosas. A latência que o compromisso de produto limita é a do pipeline inteiro no dispositivo, e ela tem quatro componentes: a carga do modelo em memória, que domina o primeiro uso e reaparece a cada reinício, o chamado cold start; o pré-processamento da entrada, redimensionar a

imagem, extrair a janela do sinal; a inferência propriamente dita; e o pós-processamento da saída. Medir apenas a inferência, que é o número que os benchmarks publicam, subestima o total, às vezes por múltiplos, quando o pré-processamento roda em CPU lenta ou quando o processo é reiniciado com frequência e paga o cold start repetidamente. O benchmark on-device do quinto passo mede o pipeline, no ciclo de uso real, e é esse número que se compara com o orçamento do tópico 1 deste ebook.

As linhas do quadro se combinam com frequência, e cada combinação é uma decisão de dois degraus, cada degrau com a sua verificação. A escolha do runtime, por sua vez, decorre da cadeia de compatibilidade montada no terceiro passo, e três situações cobrem a maior parte dos casos: runtime específico do ecossistema quando o alvo pertence àquele ecossistema, como Core ML em um dispositivo Apple; runtime neutro quando a frota é heterogênea e desacoplar treino de produção vale mais, como ONNX Runtime; e runtime especializado quando o caso é específico, como llama.cpp para um LLM pequeno em CPU. Nenhuma dessas escolhas é estética: cada uma se justifica pelo alvo, e o registro da decisão deve dizer qual característica do alvo a justificou.

Exemplo aplicado

Uma fabricante de elevadores embarca manutenção preditiva num gateway com 512 MB de memória livre, sobre séries de vibração. Em FP16, o modelo coube sem folga para as ativações nos picos. Em INT8, coube com margem e perdeu 0,4 ponto de F1 no dado real, aceito pela equipe. INT4 dobraria a folga, mas o benchmark on-device mostrou perda concentrada no slice de baixa rotação, justamente onde as falhas incipientes aparecem. Decisão registrada: INT8, com a margem restante reservada ao crescimento do modelo na próxima versão, e INT4 descartado com o número que o descartou.

⚠ ATENÇÃO

A confusão decisória desta etapa é escolher o regime de compressão pela tabela de benchmark público, do tipo INT8 perde menos de um ponto em média, e pular a validação no dado próprio. A média de um benchmark é calculada sobre outra distribuição, com outros slices e outro pior caso. A consequência é descobrir em produção uma degradação concentrada num slice que o benchmark não continha, sem registro de quando e por que o regime foi escolhido. A decisão de compressão só está tomada quando o número vem do dispositivo alvo e do dado da aplicação.

SAIBA MAIS

Para o tratamento sistemático de compressão orientada a alvo, veja HAN, S., TinyML and Efficient Deep Learning Computing (MIT 6.5940), materiais abertos em efficientml.ai. Para quantização agressiva de modelos generativos, FRANTAR, E. et al., GPTQ, em https://arxiv.org/abs/2210.17323. A otimização específica por alvo de hardware e o tuning de baixo nível são tratados nos cursos de Visão Computacional e Real Time Processing da Fase 2 do programa.

Posicionada a inferência e dimensionado o modelo para o alvo, falta o terceiro eixo, que responde o que essa configuração custa ao longo do tempo, e que muitas vezes decide os casos em que os dois primeiros eixos empatam.

3. A estrutura de custo total

Esta seção tem por objetivo que você estime, de forma qualitativa, o custo total de uma arquitetura Edge versus Cloud em regimes de baixo e de alto volume, e use essa estimativa como terceiro eixo da decisão de deployment; ela pertence à camada decisória do módulo. Os dois primeiros eixos responderam onde a inferência ocorre e como o modelo cabe nesse onde. O terceiro responde o que se paga por essa configuração, e é ele que desempata os cenários em que nenhum gatilho do tópico 1 deste ebook disparou, além de funcionar como teste de sanidade dos outros dois: uma decisão tecnicamente correta que não fecha a conta não sobrevive ao ciclo do projeto.

O conceito operacional é o custo total, e não a fatura de inferência. Comparar o preço por chamada da nuvem com o preço de compra do dispositivo é a versão mais comum da conta errada, porque ignora tudo o que os dois lados custam depois da primeira fatura e da primeira instalação.

GLOSSÁRIO

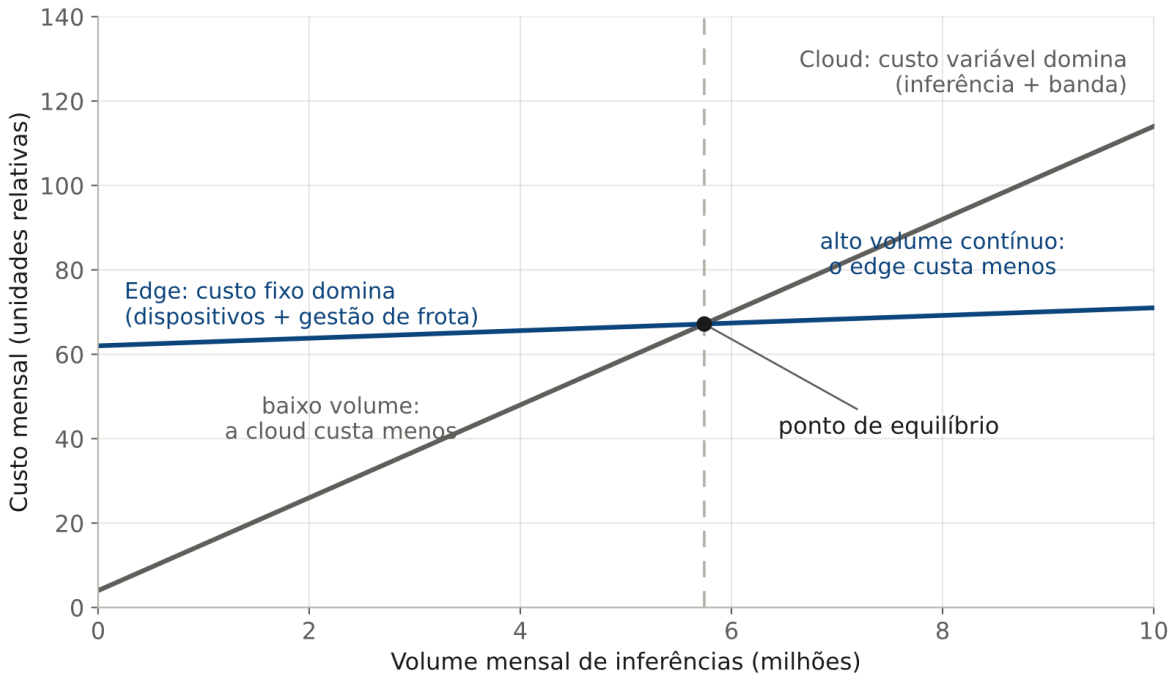
Custo total de propriedade (TCO)

Soma de tudo o que uma alternativa custa ao longo do ciclo de vida: aquisição, instalação, operação, manutenção, atualização e desativação. Na decisão Edge versus Cloud, o TCO evita a comparação enganosa entre o preço por chamada da nuvem e o preço de compra do dispositivo, que deixa de fora banda, gestão de frota, atualização de modelo e substituição de unidades.

As duas estruturas de custo têm naturezas diferentes, e é a diferença de natureza, mais do que qualquer número específico, que a decisão precisa capturar. Na cloud, o custo é majoritariamente variável: a inferência é cobrada por chamada ou por tempo de computação, a banda de subida cresce com o volume e com o peso do dado bruto, a saída de dados tem preço próprio, e a base fixa é pequena. No edge, o custo é majoritariamente fixo: os dispositivos são investimento inicial, a instalação em campo tem custo por ponto, e a operação carrega o que menos aparece nas planilhas de estimativa, a gestão da frota, a atualização remota de modelo e software, a manutenção e a substituição de unidades ao longo da vida útil. Em compensação, o custo marginal por inferência é próximo de zero: depois que o dispositivo está lá, inferir mais não custa quase nada.

Dessa diferença decorre o comportamento por regime de volume. Em baixo volume, a base fixa do edge não se dilui, e a cloud custa menos: não há CAPEX, a atualização é centralizada, e a elasticidade absorve a irregularidade da demanda. Em alto volume contínuo, banda e inferência recorrentes crescem sem teto, e o edge custa menos: o custo fixo se dilui sobre milhões de inferências que individualmente não custam nada. Entre os dois regimes existe um ponto de equilíbrio, cuja posição depende dos coeficientes do caso concreto, do peso do dado que subiria, do preço da inferência gerenciada, do custo do dispositivo e da sua vida útil. A figura a seguir ilustra a estrutura com dados simulados.

Figura 3 - Estrutura de custo mensal Edge versus Cloud por regime de volume, com o ponto de equilíbrio. Dados simulados, ilustrativos.



Fonte: autor.

Figura 3 intitulada "Estrutura de custo mensal Edge versus Cloud por regime de volume, com o ponto de equilíbrio. Dados simulados, ilustrativos." O gráfico compara o custo mensal relativo das arquiteturas Edge e Cloud em função do volume mensal de inferências. O eixo horizontal representa o volume mensal de inferências, variando de 0 a 10 milhões, enquanto o eixo vertical representa o custo mensal, em unidades relativas. Duas curvas são apresentadas. A curva cinza, correspondente ao Cloud, cresce de forma acentuada conforme aumenta o volume de inferências, indicando que os custos variáveis de processamento e transmissão de dados passam a predominar. A curva azul, correspondente ao Edge, inicia em um custo mais elevado devido ao investimento em dispositivos e gestão da infraestrutura, mas cresce lentamente à medida que o volume aumenta, refletindo a predominância de custos fixos. As duas curvas se cruzam aproximadamente na marca de 6 milhões de inferências por mês, destacada por um ponto preto e uma linha vertical tracejada, identificados como o ponto de equilíbrio. À esquerda do gráfico há a indicação de que, em baixo volume, a Cloud custa menos. À direita, uma anotação informa que, em alto volume contínuo, o Edge custa menos. O gráfico utiliza uma linha azul para representar o Edge e uma linha cinza para representar a Cloud, destacando visualmente como o comportamento dos custos muda conforme o volume de processamento aumenta. Na parte inferior consta a indicação "Fonte: autor."

A estimativa que o módulo pede é qualitativa, e quatro perguntas costumam determinar o sinal da comparação antes de qualquer planilha. O volume é contínuo ou em rajadas? Rajadas favorecem a elasticidade da cloud, que cobra só quando usada; volume contínuo favorece o custo fixo do edge. O dado que subiria é pesado, como vídeo ou áudio contínuo, ou leve, como medições esparsas? Dado pesado infla a banda e empurra para o edge. A frota seria concentrada ou espalhada? Dispersão geográfica encarece instalação e manutenção do edge. O modelo muda com que frequência? Atualização frequente favorece a cloud, onde um deploy vale para todos, contra a atualização unidade a unidade da frota. Duas dessas respostas costumam decidir o sinal; as outras ajustam a magnitude e entram na justificativa registrada.

Quadro 2 - Componentes de custo e seu comportamento em cada arquitetura.

Componente	Na cloud	No edge
Inferência	recorrente, cresce com o volume	marginal, quase zero após o investimento
Banda	subida do dado bruto, cresce com volume e peso do dado	só o derivado sobe; custo baixo e estável
Dispositivos	não há	investimento inicial mais substituição ao longo da vida útil
Atualização de modelo	um deploy central vale para todos	atualização remota unidade a unidade, com janelas e falhas
Operação	gerenciada pelo provedor	gestão de frota, monitoramento e manutenção em campo
Crescimento de volume	a fatura cresce junto, sem ação da equipe	absorvido até o limite da capacidade instalada

Quadro 2 intitulado "Componentes de custo e seu comportamento em cada arquitetura", organizado em formato de tabela com três colunas: Componente, Na cloud e No edge. O objetivo é comparar como diferentes componentes de custo se comportam em arquiteturas baseadas em computação em nuvem e em computação de borda.
A primeira linha trata da inferência. Na arquitetura em nuvem, o custo é recorrente e cresce conforme aumenta o volume de solicitações. Na arquitetura de borda, o custo por inferência é marginal, tornando-se praticamente nulo após o investimento inicial na infraestrutura.
A segunda linha apresenta a banda de comunicação. Na nuvem, ocorre o envio do dado bruto, fazendo com que o custo aumente de acordo com o volume e o tamanho dos dados transmitidos. No edge, apenas o resultado processado é enviado, mantendo um custo baixo e mais estável.
A terceira linha refere-se aos dispositivos. Na arquitetura em nuvem não há custo relacionado à aquisição de equipamentos para processamento local. Já no edge existe um investimento inicial em dispositivos, além da necessidade de substituição ao longo de sua vida útil.
Na quarta linha é abordada a atualização do modelo. Na nuvem, uma única implantação centralizada atende todos os usuários. No edge, as atualizações precisam ser distribuídas remotamente para cada dispositivo, considerando janelas de atualização e possíveis falhas durante o processo.
A quinta linha trata da operação. Na arquitetura em nuvem, a infraestrutura é gerenciada pelo provedor do serviço. No edge, a organização passa a assumir atividades como gestão da frota de dispositivos, monitoramento e manutenção em campo.
A última linha compara o crescimento do volume de uso. Na nuvem, o aumento da demanda faz a fatura crescer proporcionalmente, sem necessidade de ações diretas da equipe. No edge, o crescimento é absorvido pela capacidade instalada até seu limite, momento em que novos investimentos em infraestrutura podem ser necessários.

A comparação exige ainda fixar o horizonte temporal, porque as duas estruturas envelhecem de forma diferente. Comparar faturas mensais esconde o investimento inicial do edge; comparar apenas o investimento esconde o custo recorrente da cloud. O horizonte natural é a vida útil dos dispositivos, tipicamente de três a cinco anos em ambiente industrial: soma-se tudo o que cada alternativa custa nesse período, no volume projetado de produção, incluindo a substituição de unidades que falham antes do fim. E

o ponto de equilíbrio não é estático: o preço da inferência gerenciada tende a cair, o hardware de borda barateia e o volume do negócio muda. Uma decisão registrada com os coeficientes explícitos pode ser revisitada nos marcos do projeto quando algum coeficiente se mover de forma relevante; uma decisão intuída não oferece nada para revisitar.

O custo também pode ser dividido, e a arquitetura híbrida do tópico 1 deste ebook tem uma leitura econômica própria. O padrão mais comum coloca a inferência de alta frequência no edge, onde o custo marginal é nulo, e reserva a cloud para o que é esporádico e pesado: o retreino do modelo, o processamento dos casos difíceis que o modelo local sinaliza com baixa confiança, a agregação analítica sobre os derivados que sobem. Essa divisão captura o melhor das duas estruturas, custo fixo diluído no volume alto e elasticidade paga só quando usada, ao preço de operar duas superfícies em vez de uma. A conta do híbrido precisa incluir esse custo de coordenação, que é real ainda que não apareça em nenhuma fatura.

Uma estimativa honesta nomeia também os custos escondidos dos dois lados, para não degenerar em defesa de uma arquitetura. Do lado do edge: a heterogeneidade que a frota acumula ao longo dos anos, com dispositivos comprados em lotes e versões diferentes; a depreciação e o fim de suporte do hardware; e o custo de engenharia de manter confiável o pipeline de atualização remota, que é um sistema em si. Do lado da cloud: a saída de dados subestimada nas contas iniciais; a dependência de fornecedor, que expõe o projeto a reprecificação; e o crescimento de volume não projetado, que transforma um piloto barato numa fatura dominante sem que nenhuma decisão explícita tenha sido tomada. O registro da decisão deve citar, dos dois conjuntos, os itens relevantes ao caso.

Exemplo aplicado

Uma rede varejista mede contagem e fluxo de pessoas por câmera em 300 lojas, em operação contínua no horário comercial. O dado é vídeo, pesado, e o volume é contínuo: só a banda de subida superaria em poucos meses o custo dos dispositivos, e a inferência recorrente cresceria com a rede. A decisão de produção foi edge nas lojas, com agregados subindo de hora em hora. No piloto, porém, com 5 lojas e modelo atualizado toda semana, a mesma conta favoreceu a cloud: volume baixo, atualização frequente e nenhum compromisso de latência. O regime de volume mudou a resposta sem mudar o problema.

> ⚠ ATENÇÃO
>
> A confusão decisória desta etapa é estimar o custo no volume do piloto e decidir a arquitetura de produção com esse número. O piloto está, quase por definição, no regime de baixo volume, em que a cloud vence; a produção pode estar do outro lado do ponto de equilíbrio. A consequência é uma arquitetura que precisa ser refeita no momento de escalar, exatamente quando o cronograma tem menos folga. A comparação de custo se faz no volume projetado de produção, e o piloto é tratado como fase do projeto, não como regime permanente.

Por fim, a estrutura de custo conversa com a estrutura financeira da organização, e ignorar isso é subestimar um stakeholder real. O edge concentra investimento, o que em contabilidade é despesa de capital, e a cloud dilui pagamento recorrente, despesa operacional; organizações diferentes têm preferências diferentes entre as duas, por caixa, por regime tributário ou por política de orçamento, e essas preferências podem pesar tanto quanto o ponto de equilíbrio técnico. O papel de quem conduz a decisão não é arbitrar essa preferência, e sim apresentar a comparação nas duas leituras, o custo total no horizonte e o perfil de desembolso ao longo dele, para que a escolha registrada seja defensável também diante de quem aprova orçamento.

4. Conclusão

As três seções desta camada decisória formam uma única decisão de deployment, registrada em três eixos que se condicionam mutuamente. O posicionamento no continuum fixa onde a inferência ocorre, a partir de três gatilhos testáveis e de uma pergunta econômica. A escolha de compressão e runtime fixa como o modelo cabe nesse onde, partindo do alvo e terminando na validação no dado real. A estrutura de custo total fixa o que a configuração paga em cada regime de volume, e devolve a decisão ao primeiro eixo quando a conta não fecha, como a árvore do tópico 1 já antecipava. Nenhum eixo decide sozinho: o posicionamento define o alvo que condiciona a compressão, o regime de compressão altera a conta de custo, porque um modelo menor cabe num dispositivo mais barato, e a conta de custo pode reabrir o posicionamento.

O fio que atravessa os três eixos é o mesmo: a decisão de deployment é um compromisso de produto, e não uma otimização isolada de engenharia. Ela se condiciona com requisitos regulatórios, pelo gatilho de privacidade e soberania, e com expectativas de stakeholder, pelo SLA, pelo orçamento e pelo plano de escala, e por isso precisa estar

explícita no escopo desde o início, e não descoberta pelo time de infraestrutura no fim. É essa explicitação que o encontro presencial de desenho da solução exige: a arquitetura conceitual e a matriz de validação pressupõem que o ponto do continuum, a cadeia de formato, runtime e hardware, e a estrutura de custo estejam registrados com justificativa. Na residência, o mesmo registro sustenta qualquer projeto com restrições de latência, de conectividade ou regulatórias.

O instrumento de registro é o template T4.1, matriz de decisão Edge versus Cloud, objeto próprio da camada decisória do módulo, com os três eixos deste texto nas linhas e campo de justificativa por eixo. Preenchê-lo para um cenário concreto é a evidência de aprendizagem que o módulo pede: a capacidade avaliada desloca-se de executar uma quantização, exercitada no notebook do módulo, para justificar a decisão de deployment diante de quem vai pagar por ela e operá-la.

REFERÊNCIAS

Bibliografia mínima

HAN, S. TinyML and Efficient Deep Learning Computing (MIT 6.5940). Materiais de curso, edição 2024. Disponível em https://efficientml.ai.

JACOB, B. et al. Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference. In: Conference on Computer Vision and Pattern Recognition (CVPR), 2018. Preprint em https://arxiv.org/abs/1712.05877.

Bibliografia complementar

HAN, S.; MAO, H.; DALLY, W. J. Deep Compression: Compressing Deep Neural Networks with Pruning, Trained Quantization and Huffman Coding. In: International Conference on Learning Representations (ICLR), 2016. Preprint em https://arxiv.org/abs/1510.00149.

DETTMERS, T. et al. LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale. In: Advances in Neural Information Processing Systems (NeurIPS), 2022. Preprint em https://arxiv.org/abs/2208.07339.

FRANTAR, E. et al. GPTQ: Accurate Post-training Quantization for Generative Pre-trained Transformers. In: International Conference on Learning Representations (ICLR), 2023. Preprint em https://arxiv.org/abs/2210.17323.

GERGANOV, G. llama.cpp Project Documentation. Edição em curso. Disponível em https://github.com/ggerganov/llama.cpp.

Industr.IA

Residência em TIC 49



Executor iRede

Executor Atlântico

Coordenador Softex

Financiador MCTI Futuro

Ministério da Ciência, Tecnologia e Inovação

Governo do Brasil — Do lado do povo brasileiro

Projeto apoiado pelo Ministério da Ciência, Tecnologia e Inovações, com recursos da Lei nº 8.248, de 23 de outubro de 1991.